გაკვეთილი 8 Mod 4 (4.2)

გაკვეთილი 6 — TypeScript-ში `switch` პირობითი კონსტრუქცია

1. წინა გაკვეთილის გახსენება

მეხუთე გაკვეთილში ვისწავლეთ `if`, `else if` და `else` კონსტრუქციები.

მაგალითად:

```
let day: number = 2;

if (day === 1) {
    console.log("Monday");
} else if (day === 2) {
    console.log("Tuesday");
} else if (day === 3) {
    console.log("Wednesday");
} else {
    console.log("Unknown day");
}
```

შედეგი:

```
Tuesday
```

ეს კოდი სწორად მუშაობს, მაგრამ თუ ბევრი შესაძლო მნიშვნელობა გვაქვს, `else if` კონსტრუქცია შეიძლება საკმაოდ გრძელი გამოვიდეს.

ასეთ შემთხვევებში შეგვიძლია გამოვიყენოთ `switch`.

2. რა არის `switch`?

`switch` არის პირობითი კონსტრუქცია, რომელიც ცვლადის ან გამოსახულების მნიშვნელობას სხვადასხვა ვარიანტს ადარებს.

მისი საშუალებით შეგვიძლია ერთი მნიშვნელობის მიხედვით სხვადასხვა მოქმედება შევასრულოთ.

სინტაქსი

```
switch (value) {
    case 1:
        // მოქმედება
        break;

    case 2:
        // მოქმედება
        break;

    default:
        // მოქმედება
}
```

განვიხილოთ თითოეული ნაწილი.

ნაწილი	დანიშნულება
`switch`	იწყებს მნიშვნელობის შემოწმებას
`case`	განსაზღვრავს შესადარებელ მნიშვნელობას
`break`	წყვეტს `switch` კონსტრუქციის შესრულებას
`default`	სრულდება, თუ არცერთი `case` არ დაემთხვა

მაგალითი 1 — კვირის დღეები

```
let day: number = 3;

switch (day) {
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    case 4:
        console.log("Thursday");
        break;
```

```
    default:
        console.log("Unknown day");
}
```

შედეგი:

```
Wednesday
```

როგორ მუშაობს პროგრამა?

1. `day` ცვლადის მნიშვნელობა არის `3`.
2. `switch` ადარებს მას თითოეულ `case` მნიშვნელობას.
3. `case 3` ემთხვევა `day`-ს.
4. სრულდება `console.log("Wednesday")`.
5. `break` წყვეტს `switch`-ის შესრულებას.

3. `break` — რატომ არის მნიშვნელოვანი?

`break` გამოიყენება იმისთვის, რომ შესაბამისი `case`-ის შესრულების შემდეგ პროგრამამ `switch` დატოვოს.

მაგალითი — `break`-ის გარეშე

```
let number: number = 2;

switch (number) {
    case 1:
        console.log("One");

    case 2:
        console.log("Two");

    case 3:
        console.log("Three");

    default:
        console.log("Unknown");
}
```

შედეგი:

```
Two
Three
```

```
Unknown
```

რატომ გამოვიდა რამდენიმე შედეგი?

იმიტომ, რომ `case 2`-ის შესრულების შემდეგ `break` არ გვქონდა.

პროგრამა აგრძელებს მომდევნო ინსტრუქციების შესრულებას, მიუხედავად იმისა, ემთხვევა თუ არა შემდეგი `case`-ის მნიშვნელობა.

ამ მოვლენას Fall-through ეწოდება.

სწორი ვარიანტი

```
let number: number = 2;

switch (number) {
    case 1:
        console.log("One");
        break;

    case 2:
        console.log("Two");
        break;

    case 3:
        console.log("Three");
        break;

    default:
        console.log("Unknown");
}
```

შედეგი:

```
Two
```

დასამახსოვრებელი: თუ არ გვინდა მომდევნო `case`-ების კოდის შესრულება, გამოვიყენოთ `break`.

4. `default` — როდესაც არცერთი პირობა არ სრულდება

`default` გამოიყენება იმ შემთხვევაში, როდესაც არცერთი `case` არ ემთხვევა მოცემულ მნიშვნელობას.

მაგალითად:

```
let month: number = 15;

switch (month) {
    case 1:
        console.log("January");
        break;

    case 2:
        console.log("February");
        break;

    case 3:
        console.log("March");
        break;

    default:
        console.log("Invalid month");
}
```

შედეგი:

```
Invalid month
```

`default` სავალდებულო არ არის, მაგრამ სასურველია მისი გამოყენება მაშინ, როდესაც შეიძლება მოულოდნელი მნიშვნელობა მივიღოთ.

მაგალითად, მომხმარებელმა შეიძლება შეიყვანოს `15`, მაშინ როცა თვეების რაოდენობა მხოლოდ 12-ია.

5. რამდენიმე `case` -ის გაერთიანება

ზოგჯერ რამდენიმე სხვადასხვა მნიშვნელობის შემთხვევაში ერთი და იგივე მოქმედების შესრულება გვჭირდება.

მაგალითად, კვირის დღეების შემოწმებისას გვინდა გავიგოთ, სამუშაო დღეა თუ დასვენების.

```
let day: number = 6;

switch (day) {
    case 1:
    case 2:
```

```
    case 3:
    case 4:
    case 5:
        console.log("Working day");
        break;

    case 6:
    case 7:
        console.log("Weekend");
        break;

    default:
        console.log("Invalid day");
}
```

შედეგი:

```
Weekend
```

აქ `case 6` და `case 7` ერთ მოქმედებას ასრულებენ.

ეს საშუალებას გვაძლევს, თავიდან ავიცილოთ ერთი და იმავე კოდის რამდენჯერმე დაწერა.

კიდევ ერთი მაგალითი — მომხმარებლის როლი

```
let role: string = "moderator";

switch (role) {
    case "admin":
    case "moderator":
        console.log("Management access");
        break;

    case "user":
        console.log("User access");
        break;

    default:
        console.log("Unknown role");
}
```

შედეგი:

```
Management access
```

6. `switch` და `if/else` — რა განსხვავებაა?

ორივე კონსტრუქცია გამოიყენება გადაწყვეტილების მისაღებად, მაგრამ განსხვავებული შემთხვევებისთვის არის მოსახერხებელი.

`if/else` მაგალითი

```
let score: number = 85;

if (score >= 90) {
    console.log("Excellent");
} else if (score >= 75) {
    console.log("Good");
} else {
    console.log("Passed");
}
```

აქ ვამოწმებთ რიცხვით დიაპაზონებს.

`switch` მაგალითი

```
let grade: string = "B";

switch (grade) {
    case "A":
        console.log("Excellent");
        break;

    case "B":
        console.log("Good");
        break;

    case "C":
        console.log("Passed");
        break;

    default:
        console.log("Unknown grade");
}
```

აქ ვამოწმებთ კონკრეტულ მნიშვნელობებს.

`if/else`	`switch`
მოსახერხებელია რთული პირობებისთვის	მოსახერხებელია კონკრეტული მნიშვნელობებისთვის
შეიძლება გამოვიყენოთ `>`, `<`, `>=`	`case` ძირითადად ზუსტ მნიშვნელობას ადარებს
მარტივია სხვადასხვა ცვლადის შემოწმება	ხშირად ერთ გამოსახულებას ამოწმებს
მოსახერხებელია დიაპაზონებისთვის	მოსახერხებელია მენიუსა და სტატუსებისთვის

როდის რომელი გამოვიყენოთ?

თუ გვინდა შევამოწმოთ, არის თუ არა ასაკი 18-ზე მეტი, უმჯობესია `if`.

თუ გვინდა მომხმარებლის როლის მიხედვით კონკრეტული მოქმედების შესრულება, ხშირად მოსახერხებელია `switch`.

7. TypeScript-ის თავისებურება — `switch` და მონაცემთა ტიპები

JavaScript-ში `switch` მნიშვნელობებს მკაცრი ტოლობის პრინციპით ადარებს.

მაგალითად:

```
let value: number = 10;

switch (value) {
    case "10":
        console.log("Text");
        break;

    case 10:
        console.log("Number");
        break;
}
```

TypeScript `case "10"` -თან შეცდომას გვაჩვენებს, რადგან `value` არის `number`, ხოლო `"10"` — `string`.

TypeScript წინასწარ გვაფრთხილებს, რომ ეს ტიპები ერთმანეთს არ შეესაბამება.

სწორი ვარიანტი

```
let value: number = 10;

switch (value) {
    case 5:
        console.log("Five");
        break;

    case 10:
        console.log("Ten");
        break;

    default:
        console.log("Other number");
}
```

შედეგი:

```
Ten
```

რა მოხდება, თუ მნიშვნელობა ტექსტად მივიღეთ?

მაგალითად:

```
let selectedOption: string = "2";
```

თუ გვსურს მისი რიცხვებთან შედარება, შეგვიძლია გამოვიყენოთ `Number()`.

```
let selectedOption: string = "2";

let option: number = Number(selectedOption);

switch (option) {
    case 1:
        console.log("Home");
        break;

    case 2:
        console.log("Profile");
        break;

    case 3:
        console.log("Settings");
        break;
```

```
    default:
        console.log("Invalid option");
}
```

შედეგი:

```
Profile
```

აქ ერთად გამოვიყენეთ მესამე გაკვეთილში ნასწავლი Type Conversion და `switch`.

8. Literal Types — TypeScript-ის ახალი შესაძლებლობა

ახლა გადავიდეთ TypeScript-ის მნიშვნელოვან ნაწილზე.

წინა გაკვეთილებში ვისწავლეთ მონაცემთა ძირითადი ტიპები:

```
string
number
boolean
```

მაგრამ TypeScript საშუალებას გვაძლევს, ცვლადის ტიპად კონკრეტული მნიშვნელობებიც განვსაზღვროთ.

ამას Literal Type ეწოდება.

მაგალითი

```
let direction: "left" | "right";
```

ეს ნიშნავს, რომ `direction` ცვლადში შეგვიძლია შევინახოთ მხოლოდ:

```
"left"
"right"
```

მაგალითად:

```
let direction: "left" | "right" = "left";

direction = "right";
```

ორივე სწორია.

მაგრამ:

```
direction = "up";
```

TypeScript შეცდომას გვაჩვენებს.

რატომ?

იმიტომ, რომ `"up"` არ შედის დაშვებულ მნიშვნელობებში.

მაგალითი — შეკვეთის სტატუსი

```
let orderStatus: "pending" | "shipped" | "delivered";

orderStatus = "pending";
```

ამ ცვლადში მხოლოდ სამი კონკრეტული ტექსტური მნიშვნელობის შენახვა შეიძლება.

თუ დავწერთ:

```
orderStatus = "cancelled";
```

TypeScript შეცდომას გვაჩვენებს.

რატომ არის Literal Types სასარგებლო?

იმიტომ, რომ ზოგიერთ ცვლადს არ უნდა შეეძლოს ნებისმიერი ტექსტის მიღება.

მაგალითად, შეკვეთის სტატუსი შეიძლება იყოს მხოლოდ წინასწარ განსაზღვრული მნიშვნელობებიდან ერთ-ერთი.

9. Literal Types და `switch`

ახლა Literal Types გამოვიყენოთ `switch` კონსტრუქციასთან ერთად.

მაგალითი — შეკვეთის სტატუსი

```
type OrderStatus = "pending" | "shipped" | "delivered";

function showOrderStatus(status: OrderStatus): void {
    switch (status) {
        case "pending":
            console.log("Order is being prepared");
            break;
```

```
        case "shipped":
            console.log("Order is on the way");
            break;

        case "delivered":
            console.log("Order has arrived");
            break;
    }
}

showOrderStatus("shipped");
```

შედეგი:

```
Order is on the way
```

აქ რამდენიმე ახალი რამ გვაქვს.

`type` საშუალებას გვაძლევს, ტიპს საკუთარი სახელი დავარქვათ.

```
type OrderStatus = "pending" | "shipped" | "delivered";
```

ეს ნიშნავს, რომ `OrderStatus` არის ტიპი, რომელსაც მხოლოდ სამი შესაძლო მნიშვნელობა აქვს.

ფუნქციაში:

```
function showOrderStatus(status: OrderStatus): void
```

`status` პარამეტრს მხოლოდ ამ სამი მნიშვნელობიდან ერთ-ერთის მიღება შეუძლია.

მაგალითად:

```
showOrderStatus("pending");
showOrderStatus("delivered");
```

სწორია.

მაგრამ:

```
showOrderStatus("unknown");
```

TypeScript შეცდომას გვაჩვენებს.

ამ გზით პროგრამას ვზღუდავთ მხოლოდ დასაშვები მნიშვნელობებით.

10. `switch` და Type Narrowing

მეხუთე გაკვეთილში ვისწავლეთ Type Narrowing.

ეს ნიშნავს, რომ TypeScript შემოწმების საფუძველზე ცვლადის შესაძლო ტიპებს აზუსტებს.

იგივე შეგვიძლია გავაკეთოთ `switch` კონსტრუქციითაც.

მაგალითი — `number | string`

```
function processValue(value: number | string): void {
    switch (typeof value) {
        case "number":
            console.log(value * 2);
            break;

        case "string":
            console.log(value.toUpperCase());
            break;
    }
}

processValue(15);
processValue("hello");
```

შედეგი:

```
30
HELLO
```

როგორ მუშაობს?

პირველი გამოძახება:

```
processValue(15);
```

`typeof value` აბრუნებს `"number"`-ს.

ამიტომ სრულდება:

```
case "number":
    console.log(value * 2);
    break;
```

ამ ბლოკში TypeScript-მა იცის, რომ `value` არის `number`.

მეორე გამოძახება:

```
processValue("hello");
```

`typeof value` აბრუნებს `"string"`-ს.

ამიტომ სრულდება:

```
case "string":
    console.log(value.toUpperCase());
    break;
```

ამ შემთხვევაში TypeScript-მა იცის, რომ `value` არის `string`.

ეს არის Type Narrowing-ის მაგალითი `switch` კონსტრუქციით.

11. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — კვირის დღეები

მოცემულია:

```
let day: number = 5;
```

დავალება: დაწერეთ `switch` კონსტრუქცია, რომელიც რიცხვის მიხედვით გამოიტანს კვირის დღეს.

- 1 — Monday
- 2 — Tuesday
- 3 — Wednesday
- 4 — Thursday
- 5 — Friday
- 6 — Saturday
- 7 — Sunday

თუ რიცხვი 1-დან 7-მდე არ არის, გამოიტანეთ `"Invalid day"`.

სწორი პასუხის ნახვა

სავარჯიშო 2 — კალკულატორი

მოცემულია:

```
let a: number = 20;
let b: number = 5;
let operation: string = "*";
```

დავალება: შექმენით კალკულატორი `switch` კონსტრუქციით.

ოპერაციები:

- `"+"` — შეკრება;
- `"-"` — გამოკლება;
- `"*"` — გამრავლება;
- `"/"` — გაყოფა.

თუ ოპერაცია უცნობია, გამოიტანეთ `"Invalid operation"`.

გაყოფისას შეამოწმეთ, რომ გამყოფი არ იყოს ნული.

სწორი პასუხის ნახვა

სავარჯიშო 3 — რამდენიმე `case`-ის გაერთიანება

მოცემულია:

```
let month: number = 8;
```

დავალება: შექმენით პროგრამა, რომელიც თვის ნომრის მიხედვით გამოიტანს სეზონს.

- 12, 1, 2 — Winter
- 3, 4, 5 — Spring
- 6, 7, 8 — Summer
- 9, 10, 11 — Autumn

თუ თვის ნომერი არასწორია, გამოიტანეთ `"Invalid month"`.

სწორი პასუხის ნახვა

სავარჯიშო 4 — Literal Types

შექმენით `UserRole` ტიპი, რომელიც მიიღებს მხოლოდ სამ მნიშვნელობას:

```
"admin"
"moderator"
"user"
```

შემდეგ შექმენით `role` ცვლადი და `switch`-ით შეამოწმეთ მისი მნიშვნელობა.

- `"admin"` — გამოიტანეთ `"Full access"`;
- `"moderator"` — გამოიტანეთ `"Limited access"`;
- `"user"` — გამოიტანეთ `"Basic access"`.

სწორი პასუხის ნახვა

სავარჯიშო 5 — Type Narrowing

მოცემულია ფუნქცია:

```
function checkData(data: number | string | boolean): void {

}
```

დავალება: შეავსეთ ფუნქცია ისე, რომ:

1. თუ `data` არის `number`, გამოიტანოს რიცხვის კვადრატი.
2. თუ `data` არის `string`, გამოიტანოს ტექსტი დიდი ასოებით.
3. თუ `data` არის `boolean`, გამოიტანოს საპირისპირო ლოგიკური მნიშვნელობა.

გამოიყენეთ `switch (typeof data)`.

ფუნქცია გამოიძახეთ:

```
checkData(5);
checkData("hello");
checkData(true);
```

მოსალოდნელი შედეგი:

```
25
HELLO
false
```

12. დამატებითი პრაქტიკული დავალება — შეკვეთის მართვის სისტემა

შექმენით TypeScript პროგრამა, რომელიც ონლაინ მაღაზიის შეკვეთის სტატუსს ამოწმებს.

პროგრამაში გამოიყენეთ შემდეგი ტიპი:

```
type OrderStatus =
    | "pending"
    | "processing"
    | "shipped"
    | "delivered"
    | "cancelled";
```

პროგრამამ უნდა:

1. შექმნას `status` ცვლადი `OrderStatus` ტიპით.
2. `switch` კონსტრუქციით შეამოწმოს სტატუსი.
3. `"pending"` შემთხვევაში გამოიტანოს `"Order received"`.
4. `"processing"` შემთხვევაში გამოიტანოს `"Preparing order"`.
5. `"shipped"` შემთხვევაში გამოიტანოს `"Order is on the way"`.
6. `"delivered"` შემთხვევაში გამოიტანოს `"Order completed"`.
7. `"cancelled"` შემთხვევაში გამოიტანოს `"Order cancelled"`.

დამატებით, შექმენით `isCompleted` ცვლადი, რომელიც `true` იქნება მხოლოდ მაშინ, როდესაც სტატუსი `"delivered"`-ია.